

IBMFezNetSynGTData_2_3_hops

January 27, 2026

1 Fez Network: Syndrome and Ground Truth Data Generation

This notebook generates both syndrome measurements (multi-hop paths) and ground truth error rates (single-hop edges) for the quantum network.

Outputs: - `measurements.pkl` - Syndrome measurements for all paths - `graph.pkl` - Network topology graph - `ground_truth.pkl` - Bit flip and phase flip error rates for each edge

```
[1]: from qiskit import QuantumCircuit, ClassicalRegister, transpile
import rustworkx
from rustworkx.visualization import graphviz_draw

import sys
import numpy as np
from pathlib import Path
from datetime import datetime
import pickle
import networkx as nx
from typing import List, Dict, Tuple

# Add parent directory to path for network_emulation package
sys.path.insert(0, '../..')

# Import from network_emulation package
from network_emulation import (
    # Node utilities
    Node, NODE_NAMES, NUM_NODES, INITIAL_STATES, STATE_LABELS,
    CONNECTION_TUPLES, get_node_name, route_name, get_node_id,
    # Config loading
    load_nodes_config, load_routes_config,
    # Circuit building
    build_swapping_circuit, build_multihop_swapping_circuit,
    # Experiment execution
    initialize_backend, run_provided_circuit, execute_transpiled_circuit,
    # Visualization & export
    plot_syndrome_matrix, plot_all_states_comparison, plot_state_averages_bar,
    print_matrix_summary, print_full_comparison, save_results_to_json
)
```

1.1 Configuration

```
[2]: # =====
# EXPERIMENT CONFIGURATION
# =====

USE_REAL_HARDWARE = True # Toggle between real hardware and simulator

# Experiment parameters
NUM_SHOTS = 4096          # Number of shots per circuit execution
OPTIMIZATION_LEVEL = 1    # Transpiler optimization (0 = no optimization)
MIN_HOPS = 3              # Minimum number of hops for path finding
MAX_HOPS = 4              # Maximum number of hops for path finding

# Configuration file paths
CONFIG_DIR = Path('.././configs')
CONFIG_NUM = '2' # Configuration number to use
NODES_FILE = CONFIG_DIR / f'config_{CONFIG_NUM}_nodes.json'
ROUTES_FILE = CONFIG_DIR / f'config_{CONFIG_NUM}_routes.json'

# Input graph file
GRAPH_INPUT_FILE = Path('.././pickle_files/2x3_grid_network_graph.pkl')

# Output directory (will be created with timestamp)
OUTPUT_BASE_DIR = Path('.././pickle_files')

print("="*60)
print("EXPERIMENT CONFIGURATION")
print("="*60)
print(f"Hardware mode:      {'REAL IBM HARDWARE' if USE_REAL_HARDWARE else 'FakeFeynman SIMULATOR'}")
print(f"Shots per circuit:    {NUM_SHOTS}")
print(f"Optimization level:    {OPTIMIZATION_LEVEL}")
print(f"Min hops:              {MIN_HOPS}")
print(f"Max hops:              {MAX_HOPS}")
print(f"\nConfig files:")
print(f"  Nodes: {NODES_FILE} (exists: {NODES_FILE.exists()})")
print(f"  Routes: {ROUTES_FILE} (exists: {ROUTES_FILE.exists()})")
print(f"  Graph: {GRAPH_INPUT_FILE} (exists: {GRAPH_INPUT_FILE.exists()})")
```

```
=====
EXPERIMENT CONFIGURATION
=====
```

```
Hardware mode:      REAL IBM HARDWARE
Shots per circuit:  4096
Optimization level: 1
Min hops:           3
Max hops:           4
```

Config files:

Nodes: ../../configs/config_2_nodes.json (exists: True)

Routes: ../../configs/config_2_routes.json (exists: True)

Graph: ../../pickle_files/2x3_grid_network_graph.pkl (exists: True)

```
[3]: # =====
# LOAD CONFIGURATIONS
# =====

# Load node and route configurations from JSON files
NODES = load_nodes_config(NODES_FILE)
TRANSPORT_ROUTES = load_routes_config(ROUTES_FILE)

print("="*60)
print("LOADED CONFIGURATIONS")
print("="*60)
print(f"\nNodes ({len(NODES)} total):")
for node_id, qubits in NODES.items():
    print(f"  Node {get_node_name(node_id)}: data={qubits['data']}, "
          f"encoding={qubits['encoding']}, ancilla={qubits['ancilla']}")

print(f"\nRoutes ({len(TRANSPORT_ROUTES)} total)")
print(f"Connection pairs: {len(CONNECTION_TUPLES)}")

# =====
# INITIALIZE BACKEND
# =====

backend, simulator, coupling_map, IS_REAL_HARDWARE = initialize_backend(
    USE_REAL_HARDWARE,
    secrets_path='../secrets/personal-apikey.json',
    simulation_method="matrix_product_state"
)
```

qiskit_runtime_service._discover_account:WARNING:2026-01-27 15:55:55,934:
Loading account with the given token. A saved account will not be used.

```
=====
LOADED CONFIGURATIONS
=====
```

Nodes (6 total):

Node A: data=[96], encoding=[83, 103], ancilla=[84, 104]
Node B: data=[97], encoding=[87, 107], ancilla=[88, 108]
Node C: data=[98], encoding=[91, 111], ancilla=[92, 112]
Node D: data=[116], encoding=[101, 121], ancilla=[102, 122]
Node E: data=[117], encoding=[105, 125], ancilla=[106, 126]
Node F: data=[118], encoding=[109, 129], ancilla=[110, 130]

Routes (30 total)
Connection pairs: 30
Connected to REAL backend: ibm_fez
Qubits: 156
Status: active

1.2 Load Network Graph

```
[4]: # =====  
# LOAD NETWORK GRAPH FROM PICKLE FILE  
# =====  
  
with open(GRAPH_INPUT_FILE, 'rb') as f:  
    loaded_graph = pickle.load(f)  
  
print(f"Loaded graph type: {type(loaded_graph)}")  
print(f"Nodes: {loaded_graph.nodes() if hasattr(loaded_graph, 'nodes') else 'N/A'}")  
print(f"Edges: {list(loaded_graph.edges())}")  
  
# Get single-hop edges for ground truth measurement  
single_hop_edges = list(loaded_graph.edges())  
print(f"\nSingle-hop edges for ground truth: {single_hop_edges}")
```

Loaded graph type: <class 'networkx.classes.graph.Graph'>
Nodes: [0, 1, 2, 3, 4, 5]
Edges: [(0, 1), (0, 3), (1, 2), (1, 4), (2, 5), (3, 4), (4, 5)]

Single-hop edges for ground truth: [(0, 1), (0, 3), (1, 2), (1, 4), (2, 5), (3, 4), (4, 5)]

```
[5]: # =====  
# BUILD RUSTWORKX GRAPH FOR PATH FINDING  
# =====  
  
def node_attr(node):  
    return {'label': node[1]}  
  
network_graph = rustworkx.PyGraph()  
graph_node_indices = {}  
  
for node in NODE_NAMES.items():  
    id = network_graph.add_node(node)  
    graph_node_indices.update({node[1]: id})  
  
# Add edges from the loaded graph  
for edge in single_hop_edges:
```

```

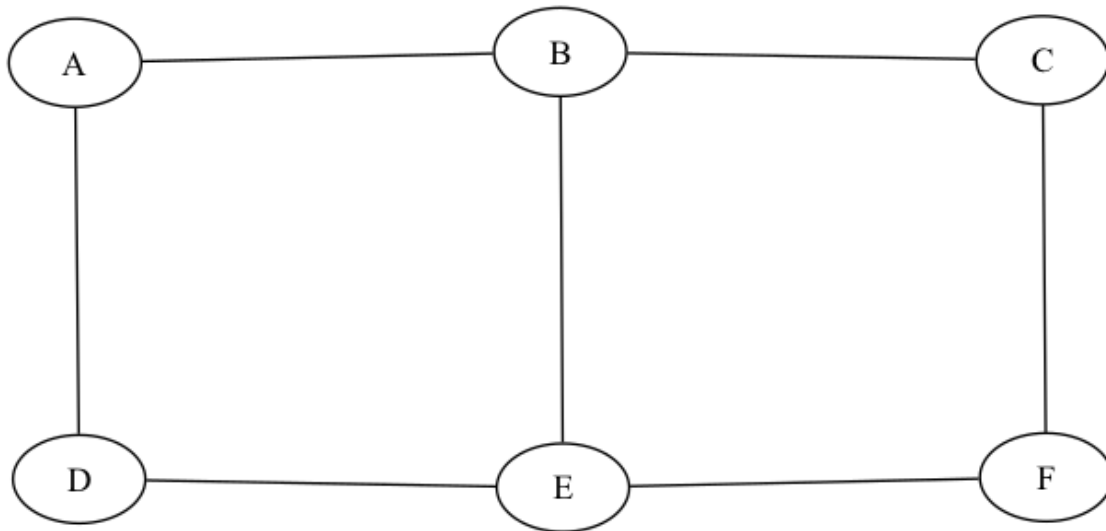
u, v = edge
# Map node IDs to rustworkx indices
network_graph.add_edge(u, v, None)

print(f"Built rustworkx graph with {len(network_graph.node_indices())} nodes_
↳and {len(network_graph.edge_list())} edges")
graphviz_draw(network_graph, node_attr_fn=node_attr, edge_attr_fn=None,
↳method='sfdp')

```

Built rustworkx graph with 6 nodes and 7 edges

[5]:



[6]:

```

# =====
# FIND ALL PATHS UP TO MAX_HOPS
# =====

PATH_DICT = {}
total_paths = 0

for node_id in network_graph.node_indices():
    node = network_graph.get_node_data(node_id)
    paths = {id: rustworkx.all_simple_paths(network_graph, node_id, to=id,
↳min_depth=MIN_HOPS, cutoff=MAX_HOPS)
    for id in network_graph.node_indices()}
    paths.pop(node_id)

    for dest_id, path_list in paths.items():
        total_paths += len(path_list)
        if path_list:

```

```

        print(f"From node {node_id} to node {dest_id}: {len(path_list)}_
        ↪paths")
        print(f"  Paths: {path_list}")

        PATH_DICT.update({node_id: paths})

print(f"\nTotal paths found: {total_paths}")

```

```

From node 0 to node 1: 1 paths
  Paths: [[0, 3, 4, 1]]
From node 0 to node 2: 1 paths
  Paths: [[0, 1, 2]]
From node 0 to node 3: 1 paths
  Paths: [[0, 1, 4, 3]]
From node 0 to node 4: 2 paths
  Paths: [[0, 3, 4], [0, 1, 4]]
From node 0 to node 5: 3 paths
  Paths: [[0, 3, 4, 5], [0, 1, 4, 5], [0, 1, 2, 5]]
From node 1 to node 0: 1 paths
  Paths: [[1, 4, 3, 0]]
From node 1 to node 2: 1 paths
  Paths: [[1, 4, 5, 2]]
From node 1 to node 3: 2 paths
  Paths: [[1, 4, 3], [1, 0, 3]]
From node 1 to node 4: 2 paths
  Paths: [[1, 2, 5, 4], [1, 0, 3, 4]]
From node 1 to node 5: 2 paths
  Paths: [[1, 4, 5], [1, 2, 5]]
From node 2 to node 0: 1 paths
  Paths: [[2, 1, 0]]
From node 2 to node 1: 1 paths
  Paths: [[2, 5, 4, 1]]
From node 2 to node 3: 3 paths
  Paths: [[2, 5, 4, 3], [2, 1, 4, 3], [2, 1, 0, 3]]
From node 2 to node 4: 2 paths
  Paths: [[2, 5, 4], [2, 1, 4]]
From node 2 to node 5: 1 paths
  Paths: [[2, 1, 4, 5]]
From node 3 to node 0: 1 paths
  Paths: [[3, 4, 1, 0]]
From node 3 to node 1: 2 paths
  Paths: [[3, 4, 1], [3, 0, 1]]
From node 3 to node 2: 3 paths
  Paths: [[3, 4, 5, 2], [3, 4, 1, 2], [3, 0, 1, 2]]
From node 3 to node 4: 1 paths
  Paths: [[3, 0, 1, 4]]
From node 3 to node 5: 1 paths
  Paths: [[3, 4, 5]]

```

```

From node 4 to node 0: 2 paths
  Paths: [[4, 3, 0], [4, 1, 0]]
From node 4 to node 1: 2 paths
  Paths: [[4, 5, 2, 1], [4, 3, 0, 1]]
From node 4 to node 2: 2 paths
  Paths: [[4, 5, 2], [4, 1, 2]]
From node 4 to node 3: 1 paths
  Paths: [[4, 1, 0, 3]]
From node 4 to node 5: 1 paths
  Paths: [[4, 1, 2, 5]]
From node 5 to node 0: 3 paths
  Paths: [[5, 4, 3, 0], [5, 4, 1, 0], [5, 2, 1, 0]]
From node 5 to node 1: 2 paths
  Paths: [[5, 4, 1], [5, 2, 1]]
From node 5 to node 2: 1 paths
  Paths: [[5, 4, 1, 2]]
From node 5 to node 3: 1 paths
  Paths: [[5, 4, 3]]
From node 5 to node 4: 1 paths
  Paths: [[5, 2, 1, 4]]

Total paths found: 48

```

1.3 Helper Functions and Classes

```

[7]: class TimeAwareMeasurement:
    """Data class for syndrome measurements with timing information."""
    def __init__(self, path_edges, histogram, duration, latency_stats=None,
        ↪code_type='513'):
        """
        :param path_edges: List of tuples representing the path, e.g., [(0,1),
        ↪(1,2)]
        :param histogram: A numpy array of syndrome counts.
        :param duration: The duration or average latency of this measurement,
        ↪(seconds).
        :param latency_stats: A dictionary with 'max', 'std', 'count'.
        :param code_type: String indicating the code type, e.g., '313_X',
        ↪'313_Z', '513'.
        """
        self.path_edges = path_edges
        self.histogram = histogram
        self.duration = duration
        self.latency_stats = latency_stats
        self.code_type = code_type

```

```

[8]: def build_ground_truth_circuit(
    node_path: List[int],

```

```

initial_state: str,
nodes: Dict[int, dict],
routes: Dict[Tuple[int, int], dict],
num_qubits: int,
measurement_basis: str = "Z",
) -> QuantumCircuit:
    """
    Build a circuit for ground truth measurement in X or Z basis.

    - Z-basis: Detects bit flips (X errors)
    - X-basis: Detects phase flips (Z errors)

    Args:
        node_path: Ordered list of node IDs describing the path (>=2 nodes).
        initial_state: "0" or "1"
        nodes: Dict mapping node_id -> {"data": int, "encoding": [int,int], ...}
        routes: Dict mapping (src_node,dst_node) -> {"movements":␣
↪ [[path1],[path2],[path3]]}
        num_qubits: Total number of physical qubits in the backend.
        measurement_basis: "X" or "Z"

    Returns:
        QuantumCircuit with measurement register
    """
    if len(node_path) < 2:
        raise ValueError("node_path must contain at least 2 nodes")
    if initial_state not in ("0", "1"):
        raise ValueError("initial_state must be '0' or '1'")
    if measurement_basis not in ("X", "Z"):
        raise ValueError("measurement_basis must be 'X' or 'Z'")

    start_node = node_path[0]
    final_node = node_path[-1]
    circuit = QuantumCircuit(num_qubits)

    src_data = nodes[start_node]["data"]
    dst_data = nodes[final_node]["data"]

    # Prepare initial state
    if initial_state == "1":
        circuit.x(src_data)

    circuit.barrier(label=f"Init: |{initial_state} @␣
↪ {get_node_name(start_node)}")

    # Rotate to measurement basis BEFORE transport
    circuit.barrier(label=f"Rotate to {measurement_basis} basis")

```

```

if measurement_basis == "X":
    circuit.h(src_data)

# Transport via SWAP operations
hopping_paths = list(zip(node_path[:-1], node_path[1:]))

for i, hop in enumerate(hopping_paths):
    src_node = hop[0]
    dst_node = hop[1]

    if (src_node, dst_node) not in routes:
        raise KeyError(f"Missing route for hop {(src_node, dst_node)}")

    route = routes[(src_node, dst_node)]["movements"]
    swapping_sequence = [list(zip(path[:-1], path[1:])) for path in route
                        if path[0] == src_data[0] and path[-1] ==
↪dst_data[0]]

    circuit.barrier(label=f"Hop {i+1}:↵
↪{get_node_name(src_node)}↪{get_node_name(dst_node)}")

    for path_swaps in swapping_sequence:
        for q1, q2 in path_swaps:
            circuit.swap(q1, q2)

# Rotate back from measurement basis
circuit.barrier(label=f"Rotate back from {measurement_basis} basis")

if measurement_basis == "X":
    circuit.h(dst_data)

# Measure
circuit.barrier(label=f"Measure @ {get_node_name(final_node)}")

creg = ClassicalRegister(1, name=f"meas_{measurement_basis}_{final_node}")
circuit.add_register(creg)
circuit.measure(dst_data, creg[0])

return circuit

```

```

[9]: def calculate_flip_error_rates(
    z_basis_counts: Dict[str, int],
    x_basis_counts: Dict[str, int],
    expected_state: str = '0'
) -> Dict[str, float]:
    """

```

```

    Calculate bit flip and phase flip error rates from X and Z basis
    measurements.

    - p_bit_flip from Z-basis measurements (Z basis detects bit flips / X
    errors)
    - p_phase_flip from X-basis measurements (X basis detects phase flips / Z
    errors)

Args:
    z_basis_counts: Measurement counts from Z-basis run
    x_basis_counts: Measurement counts from X-basis run
    expected_state: Expected outcome without errors ('0' or '1')

Returns:
    Dictionary with error rates:
    {
        'p_bit_flip': float,      # Bit flip probability (from Z-basis)
        'p_phase_flip': float,    # Phase flip probability (from X-basis)
        'total_shots': dict       # Shot counts for each basis
    }
"""

def get_error_rate(counts: Dict[str, int], correct_label: str) -> float:
    total_shots = sum(counts.values())
    if total_shots == 0:
        return 0.0
    correct_shots = counts.get(correct_label, 0)
    return 1.0 - (correct_shots / total_shots)

p_bit_flip = get_error_rate(z_basis_counts, expected_state)
p_phase_flip = get_error_rate(x_basis_counts, expected_state)

return {
    'p_bit_flip': p_bit_flip,
    'p_phase_flip': p_phase_flip,
    'total_shots': {
        'z_basis': sum(z_basis_counts.values()),
        'x_basis': sum(x_basis_counts.values())
    }
}

```

1.4 Part 1: Syndrome Measurements (Multi-hop Paths)

```

[10]: # =====
# COLLECT SYNDROME MEASUREMENTS FOR ALL PATHS
# =====

```

```

PATH_SYNDROME_METRICS = []

for src, all_paths in PATH_DICT.items():
    for dst, paths in all_paths.items():
        if not paths:
            continue
        print(f'\n\nCollecting syndrome data for {src} --> {dst}: ')

        for path in paths:
            print(f'\t{path}')

            # Build bit-flip syndrome circuit
            bit_circuit = build_multihop_swapping_circuit(
                node_path=path,
                initial_state='0',
                nodes=NODES,
                routes=TRANSPORT_ROUTES,
                num_qubits=backend.num_qubits,
                syndrome_type="bit"
            )

            # Build phase-flip syndrome circuit
            phase_circuit = build_multihop_swapping_circuit(
                node_path=path,
                initial_state='0',
                nodes=NODES,
                routes=TRANSPORT_ROUTES,
                num_qubits=backend.num_qubits,
                syndrome_type="phase"
            )

            # Execute circuits
            bit_syndrome_count = run_provided_circuit(
                circuit=bit_circuit,
                backend=backend,
                simulator=simulator,
                is_real_hardware=IS_REAL_HARDWARE,
                num_shots=NUM_SHOTS,
                optimization_level=OPTIMIZATION_LEVEL,
                return_timing=False
            )

            phase_syndrome_count = run_provided_circuit(
                circuit=phase_circuit,
                backend=backend,
                simulator=simulator,
                is_real_hardware=IS_REAL_HARDWARE,

```

```

        num_shots=NUM_SHOTS,
        optimization_level=OPTIMIZATION_LEVEL,
        return_timing=False
    )

    data = {
        'source': src,
        'dest': dst,
        'path': path,
        'bit_syn': bit_syndrome_count,
        'phase_syn': phase_syndrome_count,
        'num_shots': NUM_SHOTS,
    }
    print(f"  Bit syndrome: {bit_syndrome_count}")
    print(f"  Phase syndrome: {phase_syndrome_count}")
    PATH_SYNDROME_METRICS.append(data)

print(f"\n\nTotal syndrome measurements collected:␣
↪{len(PATH_SYNDROME_METRICS)}")

```

Collecting syndrome data for 0 --> 1:

[0, 3, 4, 1]

Job submitted to ibm_fez: d5sicnpfodos73ek2k40

Job submitted to ibm_fez: d5sicpkbmr9c739m0450

Bit syndrome: {'00': 2679, '01': 407, '10': 695, '11': 315}

Phase syndrome: {'00': 2465, '01': 442, '10': 825, '11': 364}

Collecting syndrome data for 0 --> 2:

[0, 1, 2]

Job submitted to ibm_fez: d5sics1fodos73ek2kag

Job submitted to ibm_fez: d5sictpfodos73ek2ke0

Bit syndrome: {'00': 2757, '01': 660, '10': 412, '11': 267}

Phase syndrome: {'00': 2531, '01': 770, '10': 453, '11': 342}

Collecting syndrome data for 0 --> 3:

[0, 1, 4, 3]

Job submitted to ibm_fez: d5sid2ghusoc73ept1h0

Job submitted to ibm_fez: d5sie64bmr9c739m05tg

Bit syndrome: {'00': 2484, '01': 756, '10': 494, '11': 362}

Phase syndrome: {'00': 2395, '01': 760, '10': 544, '11': 397}

Collecting syndrome data for 0 --> 4:

[0, 3, 4]

Job submitted to ibm_fez: d5siefcbmr9c739m069g
 Job submitted to ibm_fez: d5siehhfodos73ek2mg0
 Bit syndrome: {'00': 2767, '01': 377, '10': 688, '11': 264}
 Phase syndrome: {'00': 2646, '01': 408, '10': 711, '11': 331}
 [0, 1, 4]
 Job submitted to ibm_fez: d5siejscqoec73diko00
 Job submitted to ibm_fez: d5siemkbmr9c739m06i0
 Bit syndrome: {'00': 2866, '01': 353, '10': 639, '11': 238}
 Phase syndrome: {'00': 2727, '01': 398, '10': 662, '11': 309}

Collecting syndrome data for 0 --> 5:

[0, 3, 4, 5]
 Job submitted to ibm_fez: d5sieopfodos73ek2mr0
 Job submitted to ibm_fez: d5sieqhfodos73ek2mv0
 Bit syndrome: {'00': 2527, '01': 765, '10': 436, '11': 368}
 Phase syndrome: {'00': 2418, '01': 728, '10': 522, '11': 428}
 [0, 1, 4, 5]
 Job submitted to ibm_fez: d5sietkbmr9c739m06rg
 Job submitted to ibm_fez: d5sifuhfodos73ek2oag
 Bit syndrome: {'00': 2616, '01': 769, '10': 376, '11': 335}
 Phase syndrome: {'00': 2426, '01': 750, '10': 511, '11': 409}
 [0, 1, 2, 5]
 Job submitted to ibm_fez: d5sig8kcqoec73dikq4g
 Job submitted to ibm_fez: d5sigaccqoec73dikq70
 Bit syndrome: {'00': 2560, '01': 797, '10': 378, '11': 361}
 Phase syndrome: {'00': 2373, '01': 810, '10': 475, '11': 438}

Collecting syndrome data for 1 --> 0:

[1, 4, 3, 0]
 Job submitted to ibm_fez: d5sigc4cqoec73dikqa0
 Job submitted to ibm_fez: d5sigdpfodos73ek2osg
 Bit syndrome: {'00': 3006, '01': 358, '10': 452, '11': 280}
 Phase syndrome: {'00': 2794, '01': 416, '10': 529, '11': 357}

Collecting syndrome data for 1 --> 2:

[1, 4, 5, 2]
 Job submitted to ibm_fez: d5sigfkcqoec73dikqf0
 Job submitted to ibm_fez: d5sigh4bmr9c739m08p0
 Bit syndrome: {'00': 2747, '01': 479, '10': 557, '11': 313}
 Phase syndrome: {'00': 2680, '01': 449, '10': 601, '11': 366}

Collecting syndrome data for 1 --> 3:

[1, 4, 3]
 Job submitted to ibm_fez: d5sigiscqoec73dikqi0

Job submitted to ibm_fez: d5sigkghusoc73ept5kg
 Bit syndrome: {'00': 3079, '01': 291, '10': 476, '11': 250}
 Phase syndrome: {'00': 2954, '01': 351, '10': 522, '11': 269}
 [1, 0, 3]
 Job submitted to ibm_fez: d5sigmkbmr9c739m08vg
 Job submitted to ibm_fez: d5sigu8husoc73ept610
 Bit syndrome: {'00': 3098, '01': 403, '10': 349, '11': 246}
 Phase syndrome: {'00': 3036, '01': 404, '10': 394, '11': 262}

Collecting syndrome data for 1 --> 4:

[1, 2, 5, 4]
 Job submitted to ibm_fez: d5sih04cqoec73dikr30
 Job submitted to ibm_fez: d5sih2cbmr9c739m09cg
 Bit syndrome: {'00': 2747, '01': 500, '10': 467, '11': 382}
 Phase syndrome: {'00': 2489, '01': 565, '10': 591, '11': 451}
 [1, 0, 3, 4]
 Job submitted to ibm_fez: d5sih40husoc73ept6ag
 Job submitted to ibm_fez: d5sih5hfodos73ek2ppg
 Bit syndrome: {'00': 2852, '01': 478, '10': 434, '11': 332}
 Phase syndrome: {'00': 2738, '01': 455, '10': 531, '11': 372}

Collecting syndrome data for 1 --> 5:

[1, 4, 5]
 Job submitted to ibm_fez: d5sih7cbmr9c739m09l0
 Job submitted to ibm_fez: d5sih98husoc73ept6h0
 Bit syndrome: {'00': 3186, '01': 344, '10': 350, '11': 216}
 Phase syndrome: {'00': 3001, '01': 396, '10': 441, '11': 258}
 [1, 2, 5]
 Job submitted to ibm_fez: d5sihb9fodos73ek2q1g
 Job submitted to ibm_fez: d5sihd4cqoec73dikrh0
 Bit syndrome: {'00': 2980, '01': 439, '10': 373, '11': 304}
 Phase syndrome: {'00': 2867, '01': 444, '10': 443, '11': 342}

Collecting syndrome data for 2 --> 0:

[2, 1, 0]
 Job submitted to ibm_fez: d5sihkpfodos73ek2qe0
 Job submitted to ibm_fez: d5sihmcbmr9c739m0a90
 Bit syndrome: {'00': 2895, '01': 529, '10': 397, '11': 275}
 Phase syndrome: {'00': 2740, '01': 522, '10': 482, '11': 352}

Collecting syndrome data for 2 --> 1:

[2, 5, 4, 1]
 Job submitted to ibm_fez: d5sihnohusoc73ept74g
 Job submitted to ibm_fez: d5sihpghusoc73ept770

Bit syndrome: {'00': 2792, '01': 439, '10': 515, '11': 350}
Phase syndrome: {'00': 2707, '01': 494, '10': 538, '11': 357}

Collecting syndrome data for 2 --> 3:

[2, 5, 4, 3]

Job submitted to ibm_fez: d5sihr8husoc73ept79g

Job submitted to ibm_fez: d5siht0husoc73ept7c0

Bit syndrome: {'00': 2628, '01': 530, '10': 573, '11': 365}

Phase syndrome: {'00': 2560, '01': 526, '10': 606, '11': 404}

[2, 1, 4, 3]

Job submitted to ibm_fez: d5sihukcqeoc73diks70

Job submitted to ibm_fez: d5sii0ccqeoc73diks90

Bit syndrome: {'00': 2683, '01': 508, '10': 569, '11': 336}

Phase syndrome: {'00': 2608, '01': 489, '10': 627, '11': 372}

[2, 1, 0, 3]

Job submitted to ibm_fez: d5sii21fodos73ek2qu0

Job submitted to ibm_fez: d5sii3ohusoc73ept7jg

Bit syndrome: {'00': 2855, '01': 502, '10': 417, '11': 322}

Phase syndrome: {'00': 2573, '01': 595, '10': 494, '11': 434}

Collecting syndrome data for 2 --> 4:

[2, 5, 4]

Job submitted to ibm_fez: d5siibghusoc73ept7sg

Job submitted to ibm_fez: d5siid8husoc73ept7v0

Bit syndrome: {'00': 2975, '01': 380, '10': 423, '11': 318}

Phase syndrome: {'00': 2842, '01': 446, '10': 487, '11': 321}

[2, 1, 4]

Job submitted to ibm_fez: d5siigcbmr9c739m0b9g

Job submitted to ibm_fez: d5siii0husoc73ept840

Bit syndrome: {'00': 2992, '01': 457, '10': 384, '11': 263}

Phase syndrome: {'00': 2900, '01': 438, '10': 449, '11': 309}

Collecting syndrome data for 2 --> 5:

[2, 1, 4, 5]

Job submitted to ibm_fez: d5siik0husoc73ept87g

Job submitted to ibm_fez: d5siim0husoc73ept8a0

Bit syndrome: {'00': 2711, '01': 558, '10': 465, '11': 362}

Phase syndrome: {'00': 2651, '01': 501, '10': 510, '11': 434}

Collecting syndrome data for 3 --> 0:

[3, 4, 1, 0]

Job submitted to ibm_fez: d5siinohusoc73ept8cg

Job submitted to ibm_fez: d5siipghusoc73ept8eg

Bit syndrome: {'00': 2779, '01': 558, '10': 452, '11': 307}

Phase syndrome: {'00': 2545, '01': 664, '10': 491, '11': 396}

Collecting syndrome data for 3 --> 1:

[3, 4, 1]

Job submitted to ibm_fez: d5sij1kcqoec73diktj0

Job submitted to ibm_fez: d5sij3ohusoc73ept8r0

Bit syndrome: {'00': 2932, '01': 331, '10': 579, '11': 254}

Phase syndrome: {'00': 2858, '01': 349, '10': 571, '11': 318}

[3, 0, 1]

Job submitted to ibm_fez: d5sij5ccqoec73diktmg

Job submitted to ibm_fez: d5sij74cqoec73diktpg

Bit syndrome: {'00': 3030, '01': 329, '10': 480, '11': 257}

Phase syndrome: {'00': 2698, '01': 416, '10': 618, '11': 364}

Collecting syndrome data for 3 --> 2:

[3, 4, 5, 2]

Job submitted to ibm_fez: d5sij90husoc73ept920

Job submitted to ibm_fez: d5sijakcqcqoec73diktu0

Bit syndrome: {'00': 2505, '01': 618, '10': 587, '11': 386}

Phase syndrome: {'00': 2390, '01': 594, '10': 644, '11': 468}

[3, 4, 1, 2]

Job submitted to ibm_fez: d5sijckbmr9c739m0chg

Job submitted to ibm_fez: d5sijekbmr9c739m0cjg

Bit syndrome: {'00': 2648, '01': 557, '10': 572, '11': 319}

Phase syndrome: {'00': 2493, '01': 580, '10': 536, '11': 487}

[3, 0, 1, 2]

Job submitted to ibm_fez: d5sijgkbmr9c739m0cn0

Job submitted to ibm_fez: d5siji8husoc73ept9b0

Bit syndrome: {'00': 2674, '01': 583, '10': 499, '11': 340}

Phase syndrome: {'00': 2490, '01': 685, '10': 545, '11': 376}

Collecting syndrome data for 3 --> 4:

[3, 0, 1, 4]

Job submitted to ibm_fez: d5sijk9fodos73ek2ss0

Job submitted to ibm_fez: d5sijskbmr9c739m0d50

Bit syndrome: {'00': 2766, '01': 411, '10': 602, '11': 317}

Phase syndrome: {'00': 2634, '01': 507, '10': 598, '11': 357}

Collecting syndrome data for 3 --> 5:

[3, 4, 5]

Job submitted to ibm_fez: d5siju4cqoec73dikuog

Job submitted to ibm_fez: d5sijvohusoc73ept9rg

Bit syndrome: {'00': 2781, '01': 586, '10': 410, '11': 319}

Phase syndrome: {'00': 2792, '01': 535, '10': 427, '11': 342}

Collecting syndrome data for 4 --> 0:

[4, 3, 0]

Job submitted to ibm_fez: d5sik1kbmr9c739m0dag

Job submitted to ibm_fez: d5sik41fodos73ek2te0

Bit syndrome: {'00': 3110, '01': 300, '10': 430, '11': 256}

Phase syndrome: {'00': 2941, '01': 350, '10': 501, '11': 304}

[4, 1, 0]

Job submitted to ibm_fez: d5sik5ohusoc73epta2g

Job submitted to ibm_fez: d5sik7pfodos73ek2tjg

Bit syndrome: {'00': 3092, '01': 369, '10': 390, '11': 245}

Phase syndrome: {'00': 3022, '01': 410, '10': 384, '11': 280}

Collecting syndrome data for 4 --> 1:

[4, 5, 2, 1]

Job submitted to ibm_fez: d5sik9kbmr9c739m0dj0

Job submitted to ibm_fez: d5sikbkbmr9c739m0dm0

Bit syndrome: {'00': 2597, '01': 605, '10': 478, '11': 416}

Phase syndrome: {'00': 2423, '01': 546, '10': 630, '11': 497}

[4, 3, 0, 1]

Job submitted to ibm_fez: d5sikdscqoec73dikvcg

Job submitted to ibm_fez: d5sikm4bmr9c739m0e4g

Bit syndrome: {'00': 2865, '01': 487, '10': 409, '11': 335}

Phase syndrome: {'00': 2567, '01': 555, '10': 517, '11': 457}

Collecting syndrome data for 4 --> 2:

[4, 5, 2]

Job submitted to ibm_fez: d5siknpfodos73ek2u6g

Job submitted to ibm_fez: d5sikppfodos73ek2uag

Bit syndrome: {'00': 2861, '01': 394, '10': 574, '11': 267}

Phase syndrome: {'00': 2882, '01': 404, '10': 493, '11': 317}

[4, 1, 2]

Job submitted to ibm_fez: d5siks4cqoec73dikvv0

Job submitted to ibm_fez: d5siktpfodos73ek2ui0

Bit syndrome: {'00': 3219, '01': 309, '10': 385, '11': 183}

Phase syndrome: {'00': 3048, '01': 374, '10': 433, '11': 241}

Collecting syndrome data for 4 --> 3:

[4, 1, 0, 3]

Job submitted to ibm_fez: d5sikvccqoec73dil050

Job submitted to ibm_fez: d5sil14bmr9c739m0eig

Bit syndrome: {'00': 2917, '01': 473, '10': 398, '11': 308}

Phase syndrome: {'00': 2832, '01': 511, '10': 433, '11': 320}

Collecting syndrome data for 4 --> 5:

[4, 1, 2, 5]

Job submitted to ibm_fez: d5sil2ohusoc73eptb4g

Job submitted to ibm_fez: d5sil4kbmr9c739m0eo0

Bit syndrome: {'00': 2911, '01': 415, '10': 469, '11': 301}

Phase syndrome: {'00': 2668, '01': 548, '10': 482, '11': 398}

Collecting syndrome data for 5 --> 0:

[5, 4, 3, 0]

Job submitted to ibm_fez: d5sil68husoc73eptb80

Job submitted to ibm_fez: d5sil7scqoec73dil0f0

Bit syndrome: {'00': 2761, '01': 482, '10': 494, '11': 359}

Phase syndrome: {'00': 2668, '01': 541, '10': 513, '11': 374}

[5, 4, 1, 0]

Job submitted to ibm_fez: d5sil9hfodos73ek2v0g

Job submitted to ibm_fez: d5silbcbmr9c739m0f10

Bit syndrome: {'00': 2812, '01': 449, '10': 496, '11': 339}

Phase syndrome: {'00': 2675, '01': 584, '10': 489, '11': 348}

[5, 2, 1, 0]

Job submitted to ibm_fez: d5sildkbmr9c739m0f4g

Job submitted to ibm_fez: d5silf8husoc73eptbkg

Bit syndrome: {'00': 2675, '01': 534, '10': 544, '11': 343}

Phase syndrome: {'00': 2495, '01': 566, '10': 586, '11': 449}

Collecting syndrome data for 5 --> 1:

[5, 4, 1]

Job submitted to ibm_fez: d5silhccqoec73dil0r0

Job submitted to ibm_fez: d5silj4bmr9c739m0fd0

Bit syndrome: {'00': 3153, '01': 280, '10': 402, '11': 261}

Phase syndrome: {'00': 3055, '01': 295, '10': 480, '11': 266}

[5, 2, 1]

Job submitted to ibm_fez: d5sill0husoc73eptbrg

Job submitted to ibm_fez: d5silmghusoc73eptbu0

Bit syndrome: {'00': 3055, '01': 425, '10': 367, '11': 249}

Phase syndrome: {'00': 2762, '01': 458, '10': 528, '11': 348}

Collecting syndrome data for 5 --> 2:

[5, 4, 1, 2]

Job submitted to ibm_fez: d5silo9fodos73ek2vj0

Job submitted to ibm_fez: d5silqccqoec73dil16g

Bit syndrome: {'00': 2828, '01': 459, '10': 493, '11': 316}

Phase syndrome: {'00': 2588, '01': 598, '10': 527, '11': 383}

Collecting syndrome data for 5 --> 3:

[5, 4, 3]

Job submitted to ibm_fez: d5sils4bmr9c739m0fog

Job submitted to ibm_fez: d5siltsbmr9c739m0fr0

Bit syndrome: {'00': 2859, '01': 505, '10': 437, '11': 295}

Phase syndrome: {'00': 2793, '01': 483, '10': 500, '11': 320}

Collecting syndrome data for 5 --> 4:

[5, 2, 1, 4]

Job submitted to ibm_fez: d5silv9fodos73ek2vs0

Job submitted to ibm_fez: d5sim14cqoec73dil1i0

Bit syndrome: {'00': 2854, '01': 471, '10': 399, '11': 372}

Phase syndrome: {'00': 2590, '01': 563, '10': 463, '11': 480}

Total syndrome measurements collected: 48

```
[11]: # =====  
# FORMAT SYNDROME MEASUREMENTS  
# =====  
  
formatted_measurement_data = []  
  
for measurement in PATH_SYNDROME_METRICS:  
    path = measurement['path']  
    path_tuples = list(zip(path[:-1], path[1:]))  
  
    bit_syndrome = list(measurement['bit_syn'].values())  
    phase_syndrome = list(measurement['phase_syn'].values())  
  
    bit_syndrome_measurement = TimeAwareMeasurement(  
        path_edges=path_tuples,  
        histogram=np.array(bit_syndrome, dtype=np.float32),  
        duration=0.0,  
        latency_stats={},  
        code_type='313_X'  
    )  
    phase_syndrome_measurement = TimeAwareMeasurement(  
        path_edges=path_tuples,  
        histogram=np.array(phase_syndrome, dtype=np.float32),  
        duration=0.0,  
        latency_stats={},  
        code_type='313_Z'  
    )  
  
    formatted_measurement_data.append(bit_syndrome_measurement)
```

```

        formatted_measurement_data.append(phase_syndrome_measurement)

print(f"Formatted {len(formatted_measurement_data)} syndrome measurements")

```

Formatted 96 syndrome measurements

1.5 Part 2: Ground Truth Measurements (Single-hop Edges)

```

[12]: # =====
# COLLECT GROUND TRUTH ERROR RATES FOR SINGLE-HOP EDGES
# =====

edge_error_values = {}

print("Collecting ground truth measurements for single-hop edges...")
print(f"Edges to measure: {single_hop_edges}\n")

for edge in single_hop_edges:
    print(f"Processing edge: {edge}")

    # Build X-basis circuit (detects phase flips)
    circuit_x = build_ground_truth_circuit(
        node_path=edge,
        initial_state='0',
        nodes=NODES,
        routes=TRANSPORT_ROUTES,
        num_qubits=backend.num_qubits,
        measurement_basis="X"
    )

    # Build Z-basis circuit (detects bit flips)
    circuit_z = build_ground_truth_circuit(
        node_path=edge,
        initial_state='0',
        nodes=NODES,
        routes=TRANSPORT_ROUTES,
        num_qubits=backend.num_qubits,
        measurement_basis="Z"
    )

    # Transpile circuits
    transpiled_x = transpile(circuit_x, backend=backend,
    ↪optimization_level=OPTIMIZATION_LEVEL)
    transpiled_z = transpile(circuit_z, backend=backend,
    ↪optimization_level=OPTIMIZATION_LEVEL)

    # Execute circuits

```

```

print(" Executing X-basis circuit...", end=' ')
counts_x = execute_transpiled_circuit(
    transpiled_x,
    backend,
    simulator,
    num_shots=NUM_SHOTS,
    is_real_hardware=IS_REAL_HARDWARE
)
print(f"Done. Counts: {counts_x}")

print(" Executing Z-basis circuit...", end=' ')
counts_z = execute_transpiled_circuit(
    transpiled_z,
    backend,
    simulator,
    num_shots=NUM_SHOTS,
    is_real_hardware=IS_REAL_HARDWARE
)
print(f"Done. Counts: {counts_z}")

# Calculate error rates
error_rates = calculate_flip_error_rates(
    z_basis_counts=counts_z,
    x_basis_counts=counts_x,
    expected_state='0'
)

# Store as sorted edge tuple for consistency
edge_key = tuple(sorted(edge))
edge_error_values[edge_key] = {
    'p_bit_flip': error_rates['p_bit_flip'],
    'p_phase_flip': error_rates['p_phase_flip'],
    'z_counts': counts_z,
    'x_counts': counts_x
}

print(f" p_bit_flip: {error_rates['p_bit_flip']:.6f}, p_phase_flip:␣
↪{error_rates['p_phase_flip']:.6f}\n")

print("\n" + "="*60)
print("GROUND TRUTH SUMMARY")
print("="*60)
for edge, rates in edge_error_values.items():
    print(f"Edge {edge}: p_bit_flip={rates['p_bit_flip']:.6f},␣
↪p_phase_flip={rates['p_phase_flip']:.6f}")

```

Collecting ground truth measurements for single-hop edges...

Edges to measure: [(0, 1), (0, 3), (1, 2), (1, 4), (2, 5), (3, 4), (4, 5)]

Processing edge: (0, 1)

Executing X-basis circuit... Job submitted to ibm_fez: d5sim2scqoec73dil1kg
Done. Counts: {'0': 3883, '1': 213}
Executing Z-basis circuit... Job submitted to ibm_fez: d5sim4cbmr9c739m0g3g
Done. Counts: {'0': 3923, '1': 173}
p_bit_flip: 0.042236, p_phase_flip: 0.052002

Processing edge: (0, 3)

Executing X-basis circuit... Job submitted to ibm_fez: d5sim64bmr9c739m0g8g
Done. Counts: {'0': 3160, '1': 936}
Executing Z-basis circuit... Job submitted to ibm_fez: d5sim7scqoec73dil1qg
Done. Counts: {'0': 3400, '1': 696}
p_bit_flip: 0.169922, p_phase_flip: 0.228516

Processing edge: (1, 2)

Executing X-basis circuit... Job submitted to ibm_fez: d5sim9kbmr9c739m0gdg
Done. Counts: {'0': 3892, '1': 204}
Executing Z-basis circuit... Job submitted to ibm_fez: d5simbccqoec73dil1v0
Done. Counts: {'0': 3902, '1': 194}
p_bit_flip: 0.047363, p_phase_flip: 0.049805

Processing edge: (1, 4)

Executing X-basis circuit... Job submitted to ibm_fez: d5simdcbmr9c739m0gl0
Done. Counts: {'0': 3911, '1': 185}
Executing Z-basis circuit... Job submitted to ibm_fez: d5simf0husoc73eptcrg
Done. Counts: {'0': 3960, '1': 136}
p_bit_flip: 0.033203, p_phase_flip: 0.045166

Processing edge: (2, 5)

Executing X-basis circuit... Job submitted to ibm_fez: d5simh0husoc73eptcu0
Done. Counts: {'0': 3813, '1': 283}
Executing Z-basis circuit... Job submitted to ibm_fez: d5simipfodos73ek30k0
Done. Counts: {'0': 3807, '1': 289}
p_bit_flip: 0.070557, p_phase_flip: 0.069092

Processing edge: (3, 4)

Executing X-basis circuit... Job submitted to ibm_fez: d5simkscqoec73dil2b0
Done. Counts: {'0': 3453, '1': 643}
Executing Z-basis circuit... Job submitted to ibm_fez: d5simoghusoc73eptd7g
Done. Counts: {'0': 3501, '1': 595}
p_bit_flip: 0.145264, p_phase_flip: 0.156982

Processing edge: (4, 5)

Executing X-basis circuit... Job submitted to ibm_fez: d5simrhfodos73ek30ug
Done. Counts: {'0': 3863, '1': 233}
Executing Z-basis circuit... Job submitted to ibm_fez: d5simt9fodos73ek311g
Done. Counts: {'0': 3789, '1': 307}

p_bit_flip: 0.074951, p_phase_flip: 0.056885

```
=====
GROUND TRUTH SUMMARY
=====
```

```
Edge (0, 1): p_bit_flip=0.042236, p_phase_flip=0.052002
Edge (0, 3): p_bit_flip=0.169922, p_phase_flip=0.228516
Edge (1, 2): p_bit_flip=0.047363, p_phase_flip=0.049805
Edge (1, 4): p_bit_flip=0.033203, p_phase_flip=0.045166
Edge (2, 5): p_bit_flip=0.070557, p_phase_flip=0.069092
Edge (3, 4): p_bit_flip=0.145264, p_phase_flip=0.156982
Edge (4, 5): p_bit_flip=0.074951, p_phase_flip=0.056885
```

1.6 Save All Output Files

```
[13]: # =====
# CREATE OUTPUT DIRECTORY AND SAVE ALL FILES
# =====

date = datetime.today().strftime("%d-%m-%Y_%H-%M-%S")
output_path = OUTPUT_BASE_DIR / f"data_{date}"
output_path.mkdir(parents=True, exist_ok=True)

print(f"Saving outputs to: {output_path}")
print("="*60)

# -----
# Save measurements.pkl (syndrome measurements)
# -----
measurements_file = output_path / "measurements.pkl"
with open(measurements_file, "wb") as f:
    pickle.dump(formatted_measurement_data, f)
print(f"[OK] Saved measurements.pkl with {len(formatted_measurement_data)}_
    ↳samples")

# -----
# Save graph.pkl (network topology)
# -----
# Convert to networkx graph for compatibility
nx_graph = nx.Graph()
for edge in network_graph.edge_list():
    u, v = edge
    nx_graph.add_edge(u, v)

graph_file = output_path / "graph.pkl"
with open(graph_file, "wb") as f:
```

```

    pickle.dump(nx_graph, f)
print(f"[OK] Saved graph.pkl with {len(nx_graph.nodes)} nodes and {len(nx_graph.
↳edges)} edges")

# -----
# Save ground_truth.pkl (edge error rates)
# -----
ground_truth_file = output_path / "ground_truth.pkl"
with open(ground_truth_file, "wb") as f:
    pickle.dump(edge_error_values, f)
print(f"[OK] Saved ground_truth.pkl with {len(edge_error_values)} edge error_
↳rates")

print("\n" + "="*60)
print("ALL FILES SAVED SUCCESSFULLY")
print("="*60)
print(f"\nOutput directory: {output_path}")
print(f"Files:")
print(f" - measurements.pkl    ({len(formatted_measurement_data)} syndrome_
↳measurements)")
print(f" - graph.pkl           ({len(nx_graph.nodes)} nodes, {len(nx_graph.
↳edges)} edges)")
print(f" - ground_truth.pkl    ({len(edge_error_values)} edge error rates)")

```

Saving outputs to: ../../pickle_files/data_27-01-2026_16-17-51

=====

```

[OK] Saved measurements.pkl with 96 samples
[OK] Saved graph.pkl with 6 nodes and 7 edges
[OK] Saved ground_truth.pkl with 7 edge error rates

```

=====

ALL FILES SAVED SUCCESSFULLY

=====

Output directory: ../../pickle_files/data_27-01-2026_16-17-51

Files:

```

- measurements.pkl    (96 syndrome measurements)
- graph.pkl           (6 nodes, 7 edges)
- ground_truth.pkl    (7 edge error rates)

```

```

[14]: # =====
# VERIFY SAVED FILES
# =====

print("Verifying saved files...\n")

# Verify measurements.pkl

```

```

with open(measurements_file, 'rb') as f:
    loaded_measurements = pickle.load(f)
print(f"measurements.pkl: {len(loaded_measurements)} measurements")
print(f"    Sample: path_edges={loaded_measurements[0].path_edges},\n
    ↪code_type={loaded_measurements[0].code_type}")

# Verify graph.pkl
with open(graph_file, 'rb') as f:
    loaded_graph = pickle.load(f)
print(f"\ngraph.pkl: {len(loaded_graph.nodes)} nodes, {len(loaded_graph.edges)}\n
    ↪edges")
print(f"    Edges: {list(loaded_graph.edges())}")

# Verify ground_truth.pkl
with open(ground_truth_file, 'rb') as f:
    loaded_ground_truth = pickle.load(f)
print(f"\nground_truth.pkl: {len(loaded_ground_truth)} edges")
for edge, rates in loaded_ground_truth.items():
    print(f"    Edge {edge}: p_bit_flip={rates['p_bit_flip']:.6f},\n
    ↪p_phase_flip={rates['p_phase_flip']:.6f}")

```

Verifying saved files...

measurements.pkl: 96 measurements

Sample: path_edges=[(0, 3), (3, 4), (4, 1)], code_type=313_X

graph.pkl: 6 nodes, 7 edges

Edges: [(0, 1), (0, 3), (1, 2), (1, 4), (3, 4), (2, 5), (4, 5)]

ground_truth.pkl: 7 edges

Edge (0, 1): p_bit_flip=0.042236, p_phase_flip=0.052002

Edge (0, 3): p_bit_flip=0.169922, p_phase_flip=0.228516

Edge (1, 2): p_bit_flip=0.047363, p_phase_flip=0.049805

Edge (1, 4): p_bit_flip=0.033203, p_phase_flip=0.045166

Edge (2, 5): p_bit_flip=0.070557, p_phase_flip=0.069092

Edge (3, 4): p_bit_flip=0.145264, p_phase_flip=0.156982

Edge (4, 5): p_bit_flip=0.074951, p_phase_flip=0.056885